

自然语言处理导论

第六章 语言模型

目录

Contents

6.1

语言模型概述

6.2

n元语言模型

6.3

神经网络语言模型

6.4

预训练语言模型

6.5

大规模语言模型

6.6

语言模型评价方法

目录

Contents

6.1

语言模型概述

6.2

n元语言模型

6.3

神经网络语言模型

6.4

预训练语言模型

6.5

大规模语言模型

6.6

语言模型评价方法



语言模型目标是建模自然语言的概率分布，在自然语言处理研究中具有重要的作用，是机器翻译、语音识别、输入法、句法分析等任务的支撑。语言模型是自然语言处理基础任务之一，大量的研究从n元语言模型和神经语言模型等不同角度开展了系列工作。

由于语言模型可以为自然语言的表示学习提供天然的自监督训练目标，近年来，**预训练语言模型**做为通用的基于深度神经网络的自然语言处理算法的基础工具，受到越来越多的重视。大规模的预训练语言模型对于提升各类自然语言处理任务的效果起到了重要作用。



语言模型 (Language Model, LM) 目标是构建词序列 $w_1w_2...w_m$ 的概率分布 $P(w_1w_2...w_m)$ ，即计算给定的词序列 $w_1w_2...w_m$ 作为一个句子出现的可能性大小。

词汇表 V 上的语言模型由函数 $P(w_1w_2...w_m)$ 表示，对于任意词串 $w_1w_2...w_m \in V^+$ ，则有 $P(w_1w_2...w_m) \geq 0$ ，并且对于所有词串，函数 $P(w_1w_2...w_m)$ 满足归一化条件 $\sum_{w_1w_2...w_m \in V^+} P(w_1w_2...w_m) = 1$

由于联合概率 $P(w_1w_2...w_m)$ 的参数量十分巨大，直接计算 $P(w_1w_2...w_m)$ 非常困难。如果把 $w_1w_2...w_m$ 看作一个变量，那么它具有 $|V|^m$ 种可能。

按照《现代汉语词典（第七版）》包含7万词条，句子长度按照20个词计算，模型参数量达到 **$7.9792 * 10^9$** 的天文数字。



为了减少 $P(w_1w_2...w_m)$ 模型参数量，可以利用句子序列通常情况下从左至右的生成过程进行分解，使用链式法则得到：

$$\begin{aligned} P(w_1w_2...w_m) &= P(w_1)P(w_2|w_1)P(w_3|w_1w_2)\cdots P(w_m|w_1w_2...w_{m-1}) \\ &= \prod_{i=1}^m P(w_i|w_1w_2\cdots w_{i-1}) \end{aligned}$$

由此， $w_1w_2...w_m$ 的生成过程可以看作单词逐个生成的过程。首先生成 w_1 ，之后根据 w_1 生成 w_2 ，再根据 w_1 和 w_2 生成 w_3 ，以此类推，根据前 $m - 1$ 个单词生成最后一个单词 w_m

例如：对于句子“把努力变成一种习惯”的概率计算，使用公式6.1可以转化为：

$$\begin{aligned} P(\text{把 努力 变成 一种 习惯}) &= P(\text{把}) \times P(\text{努力}|\text{把}) \times P(\text{变成}|\text{把 努力}) \times \\ &\quad P(\text{一种}|\text{把 努力 变成}) \times P(\text{习惯}|\text{把 努力 变成 一种}) \end{aligned}$$



仅通过上述过程模型的参数量依然没有下降， $P(w_m | w_1 w_2 \dots w_{m-1})$ 的参数量依然是天文数字。然而基于上述转换，我们可以进一步的对模型进行简化

- n 元语言模型
- 基于分布式表示和神经网络的语言模型

语言模型的训练过程虽然采用的有监督方法，但是由于训练目标可以通过原始文本直接获得，从而使得模型的训练仅需要大规模无标注文本即可。语言模型也成为了典型的**自监督学习（Self-supervised Learning）**任务。

目录

Contents

6.1

语言模型概述

6.2

n元语言模型

6.3

神经网络语言模型

6.4

预训练语言模型

6.5

大规模语言模型

6.6

语言模型评价方法



语言模型通常用于反映一个句子出现的可能性，给定由单词序列 $w_1w_2...w_n$ 组成的句子 S ，可以利用语言的特性，使用链式法分解则得到：

$$P(S) = \prod_{i=1}^n P(w_i | w_1w_2...w_{i-1})$$

其中，词 w_i 出现的概率受它前面的 $i - 1$ 个词 $w_1w_2...w_{i-1}$ 影响，我们将这 $i - 1$ 个词 $w_1w_2...w_{i-1}$ 称之为词 w_i 的**历史**。如果历史单词有 $i - 1$ 个，那么可能的单词组合就有 $|V|^{i-1}$ 种。最简单的根据语料库对 $P(w_i | w_1w_2...w_{i-1})$ 进行估计的方法是基于词序列在语料中出现次数（也称为频次）的方法：

$$P(w_i | w_1w_2...w_{i-1}) = \frac{C(w_1w_2...w_{i-1}w_i)}{C(w_1w_2...w_{i-1})}$$



随着历史单词数量增多，绝大多数的历史并不会在训练数据中出现，这也意味着 $P(w_i|w_1w_2...w_{i-1})$ 就很可能为 0，使得概率估计失去了意义。为了解决上述问题，可以进一步假设任意单词 w_i 出现的概率只与过去 $n - 1$ 个词相关

$$P(w_i|w_1w_2...w_{i-1}) = P(w_i|w_{i-(n-1)}w_{i-(n-2)}...w_{i-1})$$

$$P(w_i|w_1^{i-1}) = P(w_i|w_{i-n+1}^{i-1})$$

满足上述条件的模型被称为**n元语法或n元文法(n-gram)**模型。其中 n-gram 表示 n 个连续单词构成的单元，也被称为**n元语法单元**。



随着历史单词数量增多，绝大多数的历史并不会在训练数据中出现，这也意味着 $P(w_i|w_1w_2...w_{i-1})$ 就很可能为 0，使得概率估计失去了意义。为了解决上述问题，可以进一步假设任意单词 w_i 出现的概率只与过去 $n - 1$ 个词相关

$$P(w_i|w_1w_2...w_{i-1}) = P(w_i|w_{i-(n-1)}w_{i-(n-2)}...w_{i-1})$$

$$P(w_i|w_1^{i-1}) = P(w_i|w_{i-n+1}^{i-1})$$

满足上述条件的模型被称为**n元语法或n元文法(n-gram)**模型。其中 n-gram 表示 n 个连续单词构成的单元，也被称为**n元语法单元**。

以二元语法为例，一个词的概率只依赖于前一个词，则句子 S 的出现概率可以表示为：

$$P(S) = \prod_{i=1}^n P(w_i|w_{i-1})$$



尽管 n 元语言模型能缓解句子概率为 0 的问题，但语言是由人和时代创造的，具备无穷的可能性，再庞大的训练语料也无法覆盖所有的 n -gram，而训练语料中的零频率并不代表零概率。因此，需要使用平滑技术（Smoothing）来解决这一问题

平滑是指为了产生更合理的概率，对最大似然估计进行调整的一类方法，也称为数据平滑（Data Smoothing）。**平滑处理的基本思想**是提高低概率，降低高概率，使整体的概率分布趋于均匀。



加法平滑 (Additive Smoothing) 是实际运用中最常用的平滑技术之一。其思想是假设事件出现的次数比实际出现的次数多 δ 次。

以二元语法模型为例，其平滑后的条件概率为：

$$P(w_i|w_{i-1}) = \frac{\delta + C(w_{i-1}w_i)}{\sum_{w_i} \delta + C(w_{i-1}w_i)} = \frac{\delta + C(w_{i-1}w_i)}{\delta|\mathbb{V}| + C(w_{i-1})}$$

其中 $0 \leq \delta \leq 1$ ， $\mathbb{V}$ 是训练语料中所有单词的集合， $C(w_i)$ 表示单词 w_i 出现的次数， $C(w_{i-1}w_i)$ 代表词 w_{i-1} 和词 w_i 同时出现次数。可以进一步拓展到 n 元语言模型上：

$$P(w_i|w_{i-n+1}^{i-1}) = \frac{\delta + C(w_{i-n+1}^i)}{\delta|\mathbb{V}| + \sum_{w_i} C(w_{i-n+1}^i)}$$

当 $\delta = 1$ 时，该方法又称为加一平滑。



古德-图灵估计法是1953年由I.J.Good基于图灵(Turning)的方法提出的，是多种平滑技术的核心。该方法基于的核心思想是将一部分已知事件的概率分配给未见的事件。

对于n元语言模型来说，降低出现次数多的n-gram，同时将剩余概率分配给未出现的n-gram。具体来说，对于任意一个出现了r次的n元语法，按照如下公式修改为出现了 r^* 次：

$$r^* = (r + 1) \frac{n_{r+1}}{n_r}$$

其中， n_r 代表有 n_r 个 n-gram 在训练语料中出现了 r 次。对其进行归一化后，即可得到出现 r 次的 n 元语法概率：

$$p_r = \frac{r^*}{N}$$

$\sum_{r>0} n_r p_r = 1 - \frac{n_1}{N} < 1$ 对于 $r = 0$ 的未见事件，有 n_1/N 的概率余量可以用于分配



表 6.1 Good-Turing 估计方法样例

r	n_r	r^*	p_r
1	3286	0.448	5.220×10^{-5}
2	736	1.25	1.454×10^{-4}
3	306	2.25	2.620×10^{-4}
4	172	3.17	3.693×10^{-4}
5	109	4.18	4.875×10^{-4}
6	76	5.53	6.440×10^{-4}
7	60	5.73	6.681×10^{-4}
8	43	5.86	6.830×10^{-4}
9	28	7.14	8.324×10^{-4}
10	20	—	—



Katz平滑是1987年由S. M. Katz所提出的后备（back-off）平滑方法，其在古德-图灵估计法的基础上引入了高阶模型与低阶模型的结合。**Katz平滑法的基本思想**是将因减值获得的概率余量根据低阶模型的分布分配给未见事件，而不是进行平均分配，从而令低概率事件有更合理的概率分布。

Katz平滑法的做法是，当事件在样本中出现的**频次大于某一数值k**时，运用最大似然估计法，通过减值来估计其概率值；而当事件的**频次小于k**值时，使用低阶的语法模型作为代替高阶语法模型的后备。

$$C_{katz}(w_{i-1}^i) = \begin{cases} d_r \frac{C(w_{i-1}w_i)}{C(w_{i-1})}, & r > 0 \\ a(w_{i-1})P_{ML}(w_i), & r = 0 \end{cases}$$



Katz平滑是1987年由S. M. Katz所提出的后备（back-off）平滑方法，其在古德-图灵估计法的基础上引入了高阶模型与低阶模型的结合。**Katz平滑法的基本思想**是将因减值获得的概率余量根据低阶模型的分布分配给未见事件，而不是进行平均分配，从而令低概率事件有更合理的概率分布。

Katz平滑法的做法是，当事件在样本中出现的**频次大于某一数值k**时，运用最大似然估计法，通过减值来估计其概率值；而当事件的**频次小于k**值时，使用低阶的语法模型作为代替高阶语法模型的后备。

$$C_{katz}(w_{i-1}^i) = \begin{cases} d_r \frac{C(w_{i-1}w_i)}{C(w_{i-1})}, & r > 0 \\ a(w_{i-1})P_{ML}(w_i), & r = 0 \end{cases}$$

所有具有非零计数 r 的二元语法都根据折扣率 d_r 被减值了。从非零计数中减去的计数量，根据低一阶的分布，即一元语法模型，分配给了计数为零的二元语法。



$$a(w_{i-1}) = \frac{1 - \sum_{w_i: C(w_{i-1}w_i) > 0} P_{katz}(w_i|w_{i-1})}{\sum_{w_i: C(w_{i-1}^i) = 0} P_{ML}(w_i)} = \frac{1 - \sum_{w_i: C(w_{i-1}^i) > 0} P_{katz}(w_i|w_{i-1})}{1 - \sum_{w_i: C(w_{i-1}^i) > 0} P_{ML}(w_i)}$$

根据修正的计数计算概率 $P_{katz}(w_i|w_{i-1})$ ，需要按照如下公式进行归一化：

$$P_{katz}(w_i|w_{i-1}) = \frac{C_{katz}(w_{i-1}^i)}{\sum_{w_i} C_{katz}(w_{i-1}^i)}$$

在二元语法的基础上可以将Katz 平滑算法拓展到高阶n 元语法模型

$$P(w_i|w_{i-n+1}^{i-1}) = \begin{cases} P_{GT}(w_i|w_{i-n+1}^{i-1}), & C(w_{i-n+1}^i) > 0 \\ a(w_{i-n+1}^{i-1})P_{GT}(w_i|w_{i-n+2}^{i-1}), & C(w_{i-n+1}^i) = 0 \text{ 并且 } C(w_{i-n+1}^{i-1}) > 0 \\ P_{BF}(w_i|w_{i-n+2}^{i-1}), & C(w_{i-n+1}^{i-1}) = 0 \end{cases}$$



除了我们在上述章节介绍的平滑算法之外，研究人员们提出了有很多针对语言模型的平滑算法，包括Jelinek Mercer平滑算法、Witten-Bell平滑算法、Kneser-Ney平滑算法等。这些方法的核心思想大都可以归纳为：**如果n-gram存在则使用其本身计数，如果不存在再退后到低阶分布。可以用如下公式表示：**

$$P_{\text{smooth}}(w_i | w_{i-n+1:i-1}) = \begin{cases} \alpha(w_i | w_{i-n+1}^{i-1}), & C(w_{i-n+1}^i) > 0 \\ \gamma(w_{i-n+1}^{i-1}) P_{\text{smooth}}(w_i | w_{i-n+2}^{i-1}), & C(w_{i-n+1}^i) = 0 \end{cases}$$

在此基础上，一些平滑算法采用高阶和低阶n元语法模型的线性插值的方法，融合高阶和低阶语法的估计，如以下公式所示：

$$P_{\text{smooth}}(w_i | w_{i-n+1}^{i-1}) = \lambda P_{\text{ML}}(w_i | w_{i-n+1}^{i-1}) + (1 - \lambda) P_{\text{smooth}}(w_i | w_{i-n+2}^{i-1})$$

目录

Contents

6.1

语言模型概述

6.2

n元语言模型

6.3

神经网络语言模型

6.4

预训练语言模型

6.5

大规模语言模型

6.6

语言模型评价方法

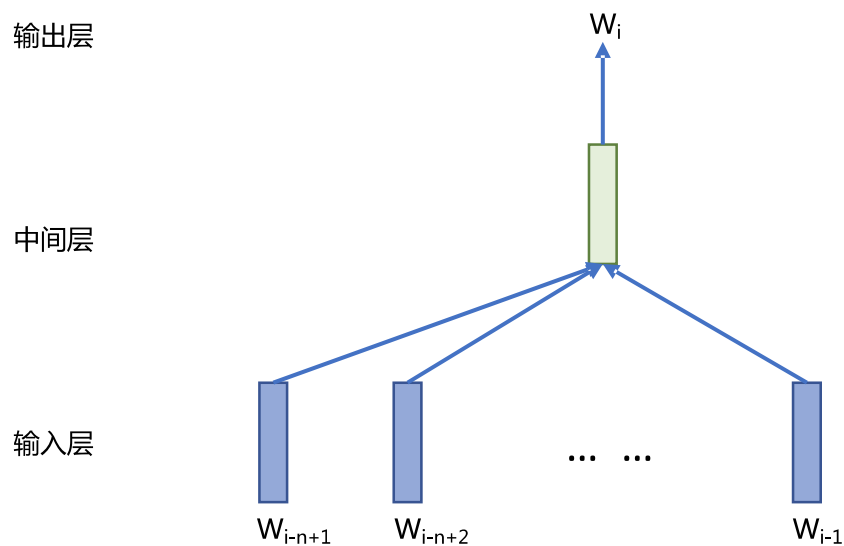


随着深度神经网络的发展，利用神经网络的语言模型展现出了比n元语言模型**更强学习能力**。神经网络先进的结构使其能有效的**建模长距离上下文依赖**，以词向量（Word Embedding）为代表的分布式表示的语言模型深刻地影响了自然语言处理领域的其他模型与应用的变革。n元语言模型几乎已经被神经网络的语言模型所替代。



给定历史单词序列 $w_1w_2\ldots w_{i-1}$ ，神经网络语言模型的目标是根据历史单词对下一时刻词进行预测。

与传统 n 元语言模型类似，前馈神经网络语言模型沿用了马尔可夫假设，即下一时刻的词只与过去 $n-1$ 个词相关，其目标可以表示为输入历史单词 $w_{i-n+1}\ldots w_{i-1}$ ，输出词 w_i 在词表 V 上的概率分布，即估计条件概率 $P(w_i|w_{(i-n+1)}^{i-1})$



输入层、隐藏层和输出层



历史词序列首先经过**输入层**被转换为离散的独热编码，随后每个词的独热编码被映射为一个低维稠密的实数向量；**隐藏层**对词向量层的输出进行编码，进行多次线性变换与非线性映射；隐藏层向量经过**输出层**被映射到词表空间，再利用Softmax函数得到其词表上的概率分布。

输入层

$$\mathbf{v} = [\mathbf{v}_{(i-n+1)}, \dots, \mathbf{v}_{i-1}]$$

$\mathbf{v}_{i-1} \in \mathbb{R}^d$ 代表词 w_{i-1} 所对应的词向量

隐藏层

$$\mathbf{h} = f(\mathbf{W}^{hid}\mathbf{v} + \mathbf{b}^{hid})$$

输出层

$$\mathbf{y} = \text{Softmax}(\mathbf{W}^{out}\mathbf{h} + \mathbf{b}^{out})$$



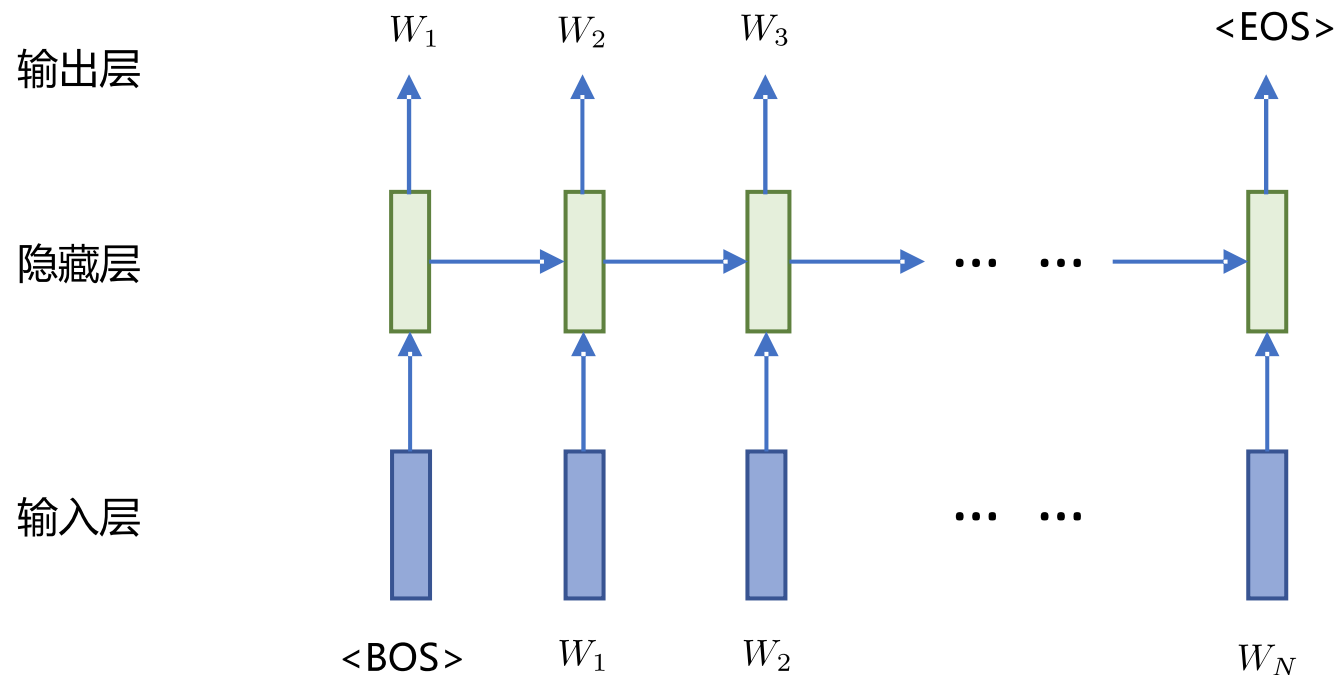
在实际场景下，固定长度的历史词并不是总能提供充分的信息，对于信息较为复杂的长文本，模型需要依赖较长的历史才能做出准确预测。

例如：与 小明 一起旅行游玩总是充满了惊喜，你永远不会知道 他 将要带你到哪里去。

循环神经网络（Recurrent Neural Network, RNN）常用于处理序列结构的数据，其特点是上一时刻的模型隐藏层状态会作为当前时刻模型的输入，每一时刻的隐藏层状态都会维护所有过去词的信息。循环神经网络语言模型不再基于马尔可夫假设，每个时刻的单词都会考虑到过去所有时刻的单词，词之间的依赖通过隐藏层状态来获取，这刚好解决了语言模型需要动态依赖的问题。



循环神经网络语言模型由三部分组成：输入层、隐藏层和输出层



$$\mathbf{x}_i = [\mathbf{v}_{i-1}; \mathbf{h}_{i-1}] \quad \mathbf{h}_i = f(\mathbf{U}\mathbf{v}_{i-1} + \mathbf{V}\mathbf{h}_{i-1} + \mathbf{b}^{hid})$$

$$\mathbf{y}_i = \text{Softmax}(\mathbf{W}^{out}\mathbf{h}_i + \mathbf{b}^{out})$$

目录

Contents

6.1

语言模型概述

6.2

n元语言模型

6.3

神经网络语言模型

6.4

预训练语言模型

6.5

大规模语言模型

6.6

语言模型评价方法



受到**计算机视觉领域**采用ImageNet对模型进行一次预选训练，使得模型可以通过海量图像充分学习如何提取特征，然后再根据任务目标进行模型精调的范式影响，自然语言处理领域基于预训练语言模型的方法也逐渐成为主流。

ELMo为代表的动态词向量模型开启了语言模型预训练的大门，此后以GPT和BERT为代表的基于Transformer的大规模预训练语言模型的出现，使得自然语言处理全面进入了**预训练微调范式**新时代。

将预训练模型应用于下游任务时，**不需要了解太多的任务细节，不需要设计特定的神经网络结构**，只需要“微调”预训练模型，即使用具体任务的标注数据在预训练语言模型上进行监督训练，就可以取得显著的性能提升。



词向量主要利用语料库中词之间的共现信息，学习词语的向量表示。因此，根据给定的语料库所学习到的**词向量是恒定不变**的，可以认为是“静态”的，不跟随上下文发生变化。然而，自然语言中词语往往具有多种语义，**在不同的上下文或语境下会具有不同的语义**。

动态词向量（Dynamic Word Embedding），也称为上下文相关的词向量（Contextualized Word Embedding方法，一个词语的向量**通过其所在的上下文计算获得，跟随上下文动态变化**。

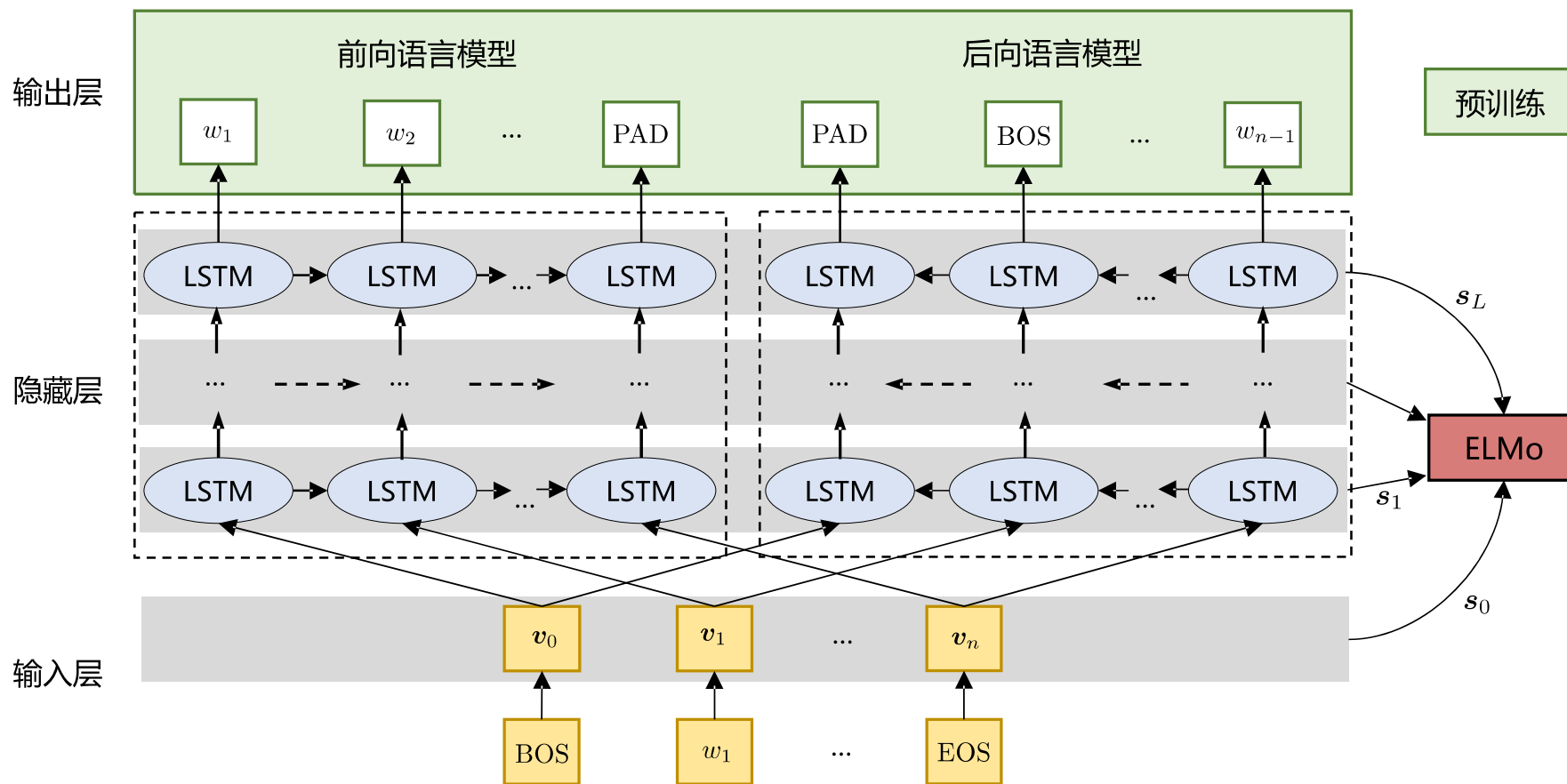


图 6.3 双向预训练语言模型 ELMo 神经网络结构



ELMo使用了两个独立的编码器分别对前向和后向进行语言模型建模，在进行预训练时，分别取最高层的正向和反向LSTM输出预测下一时刻的词。

$$P_{forward}(w_1 w_2 \dots w_n) = \prod_{i=1}^n P(w_i | w_{1:i-1}; \theta_f)$$
$$P_{backward}(w_1 w_2 \dots w_n) = \prod_{i=1}^n P(w_i | w_{i+1:n}; \theta_b)$$

ELMo算法的编码层采用了多层双向LSTM结构，通常认为，模型低层能捕捉语法等基础特征，高层能捕捉语义语境等更深层次的语言特征，双向的LSTM能保证在编码过程中每个位置都能获得该位置过去和未来位置的词信息。对于词 w_i 来说，一个L层的ELMo模型会产生 $2L+1$ 个向量表示：

$$R_i = \{v_i, \vec{h}_{i,j}, \overleftarrow{h}_{i,j} | j = 1, \dots, L\}$$



在进行下游任务时，ELMo将 R_i 中的所有向量整合成一个向量，整合的方式由任务而定，最简单的情况是直接使用最后一层的表示。因为每层LSTM学习到的信息不相同，对于不同任务来说，每层特征的重要性也不尽相同，因此更普遍的做法是根据任务所需信息，对每层的特征进行加权得到词 w_i 的对应的EMLo向量，其计算过程可以表示为：

$$\text{EMLo}_i^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} \mathbf{h}_{i,j}^{LM}$$

γ^{task} 是整体的缩放系数， s^{task} 是每层的权重系数，反映每一层向量对于目标任务的重要性。



OpenAI公司在2018年提出的GPT（Generative Pre-Training）模型是典型的生成式预训练语言模型之一。GPT-2模型结构如图6.4所示，由多层Transformer组成的单向语言模型，主要可以分为输入层，编码层和输出层三部分。

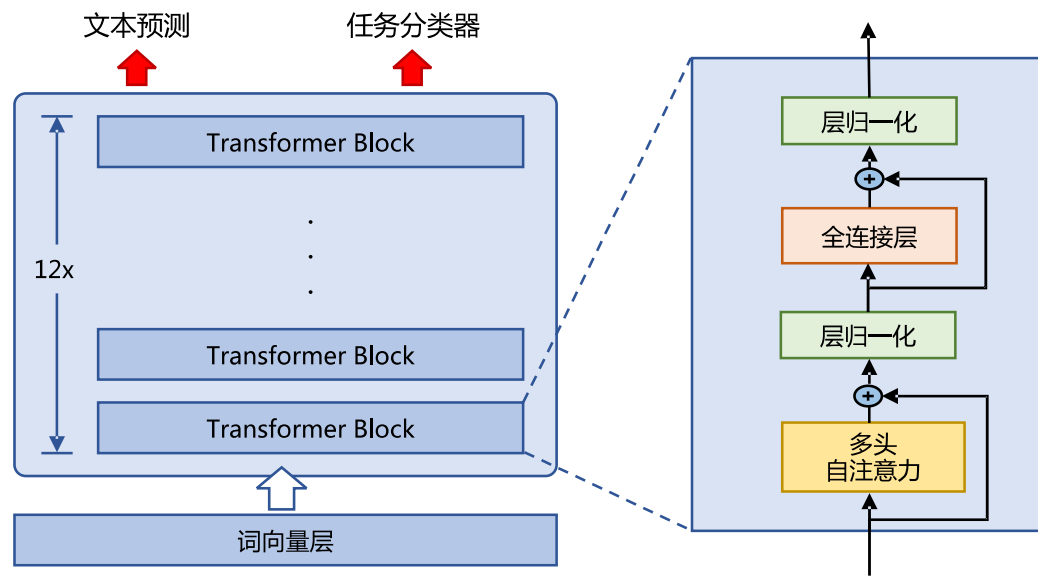


图 6.4 GPT-2 预训练语言模型结构



GPT采用生成式预训练方法，单向意味着模型只能从左到右或从右到左对文本序列建模，所采用的Transformer结构保证了输入文本每个位置只能依赖过去时刻的信息。

给定文本序列 $w = w_1 w_2 \dots w_n$ ，GPT-2 首先在输入层中将其映射为稠密的向量

$$v_i = v_i^t + v_i^p$$

v_i^t 是词 w_i 的词向量， v_i^p 是词 w_i 的位置向量， v_i 为第 i 个位置的单词经过模型输入层（第0层）后的输出

经过输入层编码，模型得到表示向量序列 $v = v_1 \dots v_n$ ，随后将 v 送入模型编码层



编码层由L个Transformer模块组成，在自注意力机制的作用下，每一层的每个表示向量都会包含之前位置表示向量的信息，使每个表示向量都具备丰富的上下文信息，并且经过多层解码后，GPT-2能得到每个单词层次化的组合式表示，其计算过程表示如下：

$$\mathbf{h}^{(L)} = \text{Transformer-Block}^{(L)}(\mathbf{h}_i^{(0)})$$

GPT-2模型的输出层基于最后一层的表示 $\mathbf{h}^{(L)}$ ，预测每个位置上的条件概率，其计算过程可以表示为：

$$P(w_i | w_1, \dots, w_{i-1}) = \text{Softmax}(\mathbf{W}^e \mathbf{h}_i^{(L)} + \mathbf{b}^{out})$$

单向语言模型}是按照阅读顺序输入文本序列 w ，用常规语言模型目标优化 w 的最大似然估计，使之能根据输入历史序列对当前词能做出准确的预测：

$$\mathcal{L}^{\text{PT}}(w) = - \sum_{i=1}^n \log P(w_i | w_0 \dots w_{i-1}; \boldsymbol{\theta})$$



通过无监督语言模型预训练，使得GPT模型具备了一定的通用语义表示能力。根据下游任务精调（Fine-tuning）的目的是在通用语义表示基础上，根据下游任务的特性进行适配。

下游任务通常需要利用有标注数据集进行训练，数据集使用D进行表示，每个样例输入长度为n的文本序列 $x=x_1x_2...x_n$ 和对应的标签y构成。

首先将文本序列x输入GPT模型，获得最后一层的最后一个词所对应的隐藏层输出 $h_n^{(L)}$ ，在此基础上通过全连接层变换结合Softmax函数，得到标签预测结果。

$$P(y|x_1...x_n) = \text{Softmax}(\mathbf{h}^{(L)} \mathbf{W}^y)$$

通过对整个标注数据集D优化如下损失函数精调下游任务：

$$\mathcal{L}^{\text{FT}}(\mathbb{D}) = \sum_{(x,y)} \log P(y|x_1...x_n)$$



下游任务在精调过程中，针对任务目标进行优化，很容易使得模型对预训练阶段所学习到的通用语义知识表示遗忘，从而损失模型的通用性和泛化能力，造成灾难性遗忘（Catastrophic Forgetting）问题。

通常会采用混合预训练任务损失和下游精调损失的方法来缓解上述问题。在实际应用中，通常采用如下公式进行下游任务精调：

$$\mathcal{L} = \mathcal{L}^{\text{FT}}(\mathbb{D}) + \lambda \mathcal{L}^{\text{PT}}(\mathbb{D})$$



2018年Devlin等人提出了掩码预训练语言模型BERT（Bidirectional Encoder Representation from Transformers）。BERT利用掩码机制构造了基于上下文预测中间词的预训练任务，相较于传统的语言模型建模方法，BERT能进一步挖掘上下文所带来的丰富语义。

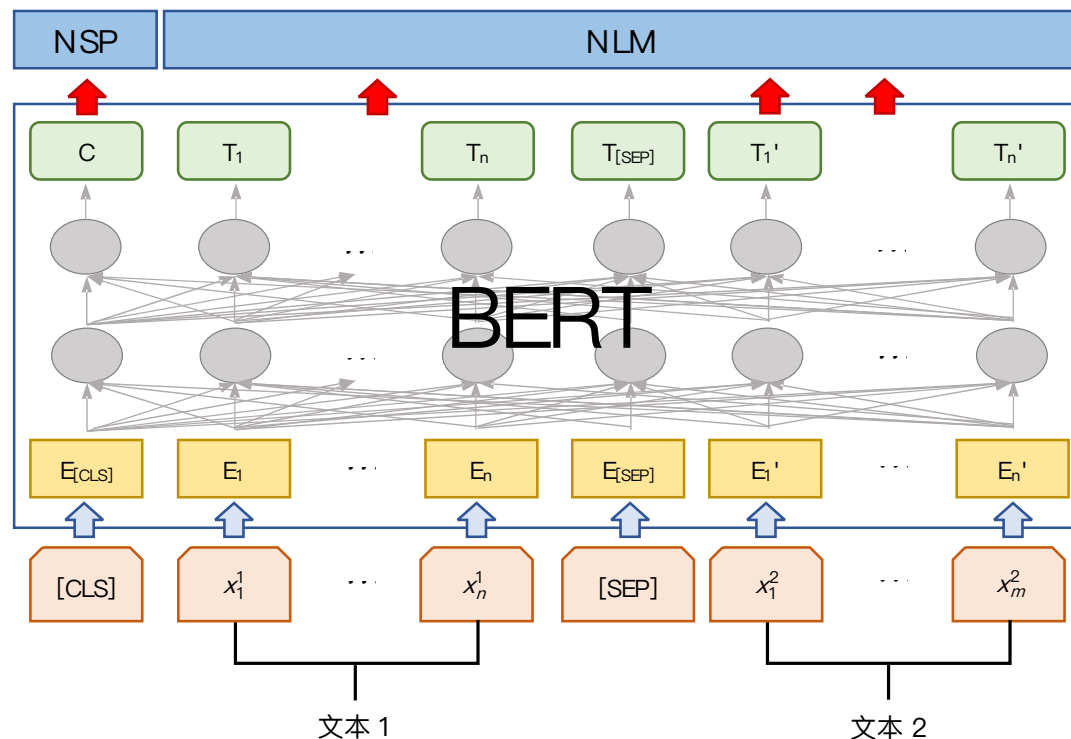


图 6.5 掩码预训练语言模型 BERT 神经网络结构



BERT由**多层Transformer编码器**组成，这意味着在编码过程中，每个位置都能获得所有位置的信息，而不仅仅是历史位置的信息。

BERT同样由输入层，编码层和输出层三部分组成。编码层由多层Transformer编码器组成。

在预训练时，模型的最后有**两个输出层MLM和NSP**，分别对应了两个不同的预训练任务：**掩码语言模型**（Masked Language Modeling, MLM）和**下一句预测**（Next Sentence Prediction, NSP）

掩码语言模型的训练对于输入形式没有要求，可以是一句话也可以一段文本，甚至可以是整个篇章，但是下一句预测则需要输入为两个句子，因此BERT在预训练阶段的输入形式统一为两段文字的拼接，这与其他预训练模型相比有较大区别。



BERT输入层采用了**WordPiece分词**，根据词频，决定是否将一个完整的词切分为多个子词以缓解OOV问题

例如：单词highest可以被切分为high和##est两个子词。

对输入文本进行分词后，BERT的输入表示由三部分组成：**词嵌入**（Token Embedding）、**段嵌入**（Segment Embedding）和**位置嵌入**（Position Embedding）。

每个词的输入表示 v 可以表示为：

$$v = v^t + v^s + v^p$$



词嵌入用来将词转换为实值向量表示。完成分词后，切分完的子词通过词嵌入矩阵转化为词嵌入表示，假设子词对应的独热向量表示为 e^t ，其对应的词嵌入 v^t ：

$$v^t = e^t W^t$$

段嵌入用于区分不同词所属的段落（Segment），同一个段落中所有词的段嵌入相同。每个段落有其特有的段编码（Segment Encoding），段编码从0开始计数。

$$v^s = e^s W^s$$

位置嵌入用于表示不同词的绝对位置。将输入序列中每个词从左到右编号后，每个词都获得位置独热编码 e^p ，通过可训练的位置嵌入矩阵 W^p 即可得到位置向量 v^p ：

$$v^p = e^p W^p$$



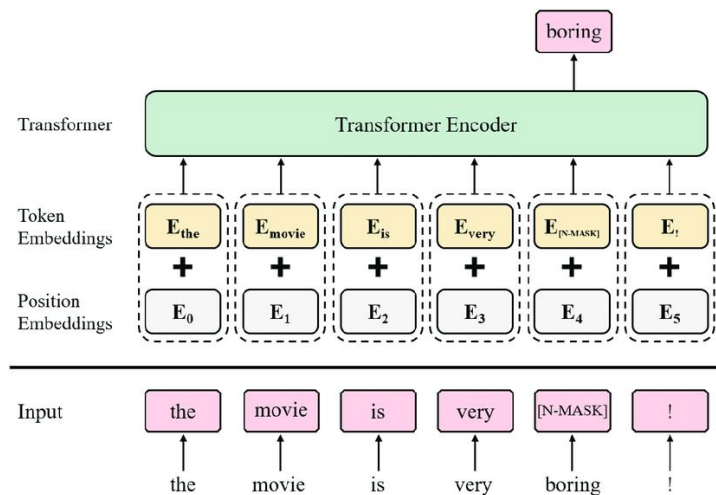
BERT的编码层采用多层Transformer结构，使用 L 表示所采用的层数， H 表示每层的隐藏单元数， A 是指自注意力头数量。在文献[29]给出了两种不同的参数设置，BERT_{BASE}使用 $L=12$ ， $H=768$ ， $A=12$ ，总参数量为110M，BERT_{LARGE}使用 $L=24$ ， $H=1024$ ， $A=16$ ，总参数量为340M。

需要注意的是，与GPT中Transformer结构所采用的约束自注意力（Constrained Self-Attention）仅关注当前单元左侧上下文不同，BERT采用的Transformer结构使用了双向多头自注意机制，不仅关注当前单元左侧上下文情况，也会关注右侧上下文。



为了更好的利用上下文信息，让当前时刻的词表示同时编码“过去”和“未来”的文本，BERT采用了一种类似于完形填空的任务，即**掩码语言建模**。在预训练时，随机将输入文本的部分单词掩盖（Mask），让模型预测被掩盖的单词，从而让模型具备根据上下文还原被掩盖的词的能力。

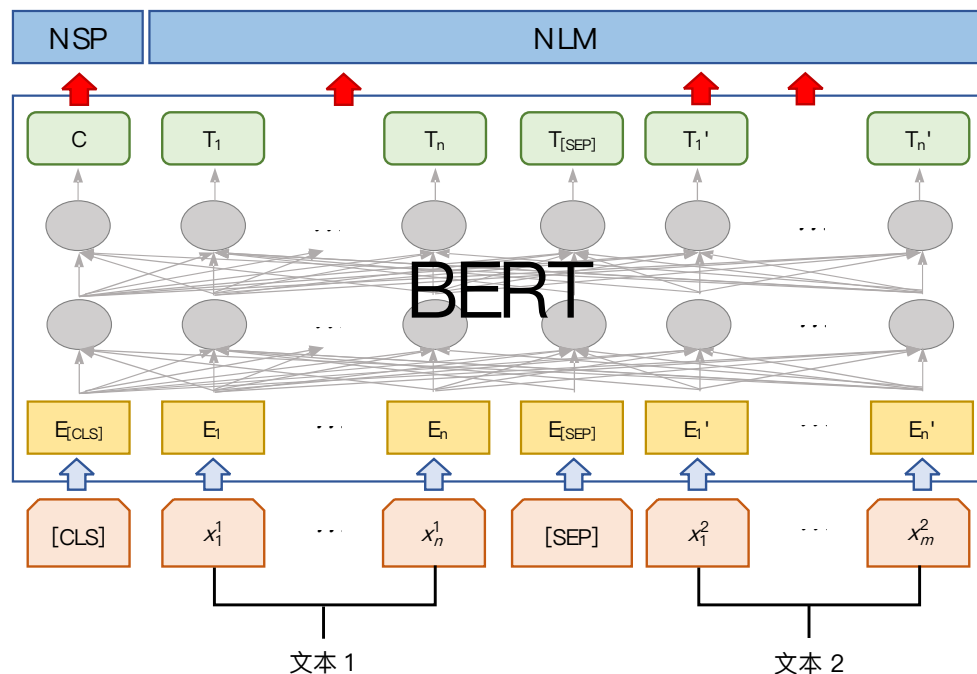
在BERT的预训练过程中，输入文本中15%的子词会被掩盖。具体来说，模型将被掩盖位置的词替换为特殊字符“[MASK]”，代表模型需要还原该位置的词。但在执行下游任务时，[MASK]字符并不会出现，这导致预训练任务和下游任务不一致。因此，在进行掩盖时，并不总是直接将词替换为[MASK]，而是根据概率从三种操作中选择一种：（1）80\%的概率替换为[MASK]；（2）10\%的概率替换为词表中任意词；（3）10\%的概率不进行替换。





对于阅读理解、语言推断等需要输入两段文本的任务来说，模型尚不具备判断两段文本关系的能力。因此，为了学习到两段文本间的关联，BERT引入了第二个预训练任务：**下一句预测**（NSP）。

下一句预测的任务目标是预测两段文本是否构成上下句的关系。对于句子A和句子B，若语料中这两个句子相邻，则构成正样本，若不相邻，则构成负样本。在预训练时，一个给定的句子对，有50%的概率将其中一句替换成来自其他段落的句子。这样可以将训练样本的正负例比例控制在1:1。





自回归式模型GPT缺乏上下文语境信息，而BERT虽然能利用上下文信息，但其预训练任务使其在自然语言生成任务上表现不佳。

BART（Bidirectional and Auto-Regressive Transformers）兼具上下文语境信息的编码器和自回归特性的解码器，配合上针对自然语言生成制定的预训练任务，使其格外契合生成任务的场景。

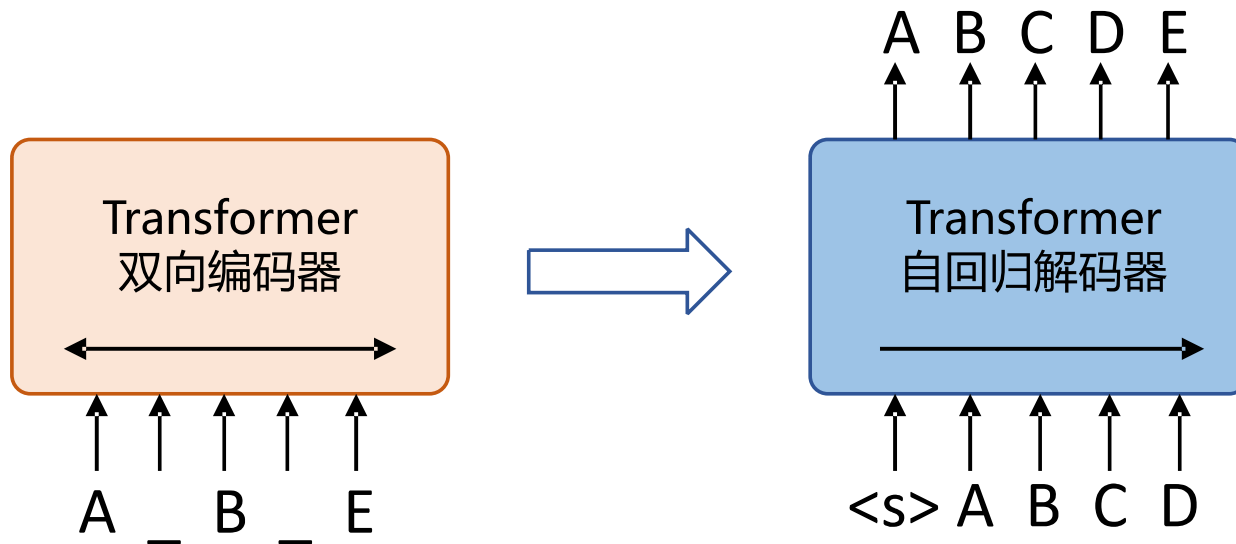


图 6.6 BART 的神经网络结构



BART的预训练过程采用的是对**含有噪声的输入文本进行去噪重构方法**，属于去噪自编码器（Denoising Autoencoder）。BART使用双向编码对引入噪声的文本进行编码，单向的自回归解码器通过自回归方式顺序重构原始文本。BART的预训练任务主要关注如何引入噪声。BART模型使用了五种方式在输入文本上引入噪音：

- **单词掩码（Token Masking）**：随机从输入文本中选择一些单词，将其替换为掩码（[MASK]）标记，类似于BERT。该噪声需要模型具备预测单个单词的能力。
- **单词删除（Token Deletion）**：随机从输入文本中删除一部分单词。该噪声除了需要模型预测单个单词的能力，还需要模型能定位缺失单词的位置。
- **文本填充（Text Infilling）**：随机将输入文本中多处连续的单词（称作文本片段）替换为一个掩码标记。文本片段的长度服从 $\lambda=3$ 的泊松分布。当文本片段长度为0时，相当于插入一个掩码标记。该噪音需要模型能识别一个文本片段有多长，并具备预测缺失片段的能力。
- **句子排列变换（Sentence Permutation）**：对于一个完整的句子，根据句号将其分割为多个子句，随机打乱子句的顺序。该噪音需要模型能一定程度上理解输入文本的语义，具备推理前后句关系的能力。
- **文档旋转变换（Document Rotation）**：随机选择输入文本中的一个单词，以该单词作为文档的开头，并旋转文档。该噪音需要模型具备找到原始文本开头的的能力。

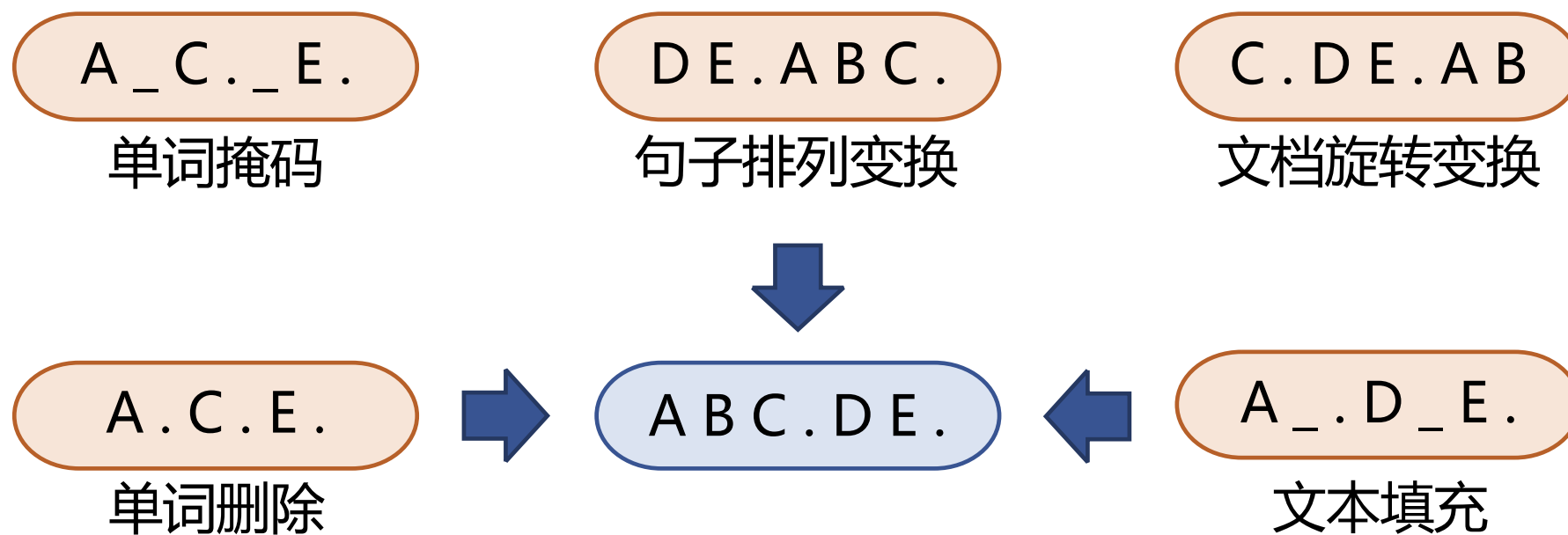


图 6.7 BART 各类型加噪方式示例



BART预训练模型具备文本表示和生成能力，因此不仅适用于文本理解任务，也适用于文本生成任务，但是用于不同类型任务时，其精调方式有所不同。

对于序列分类任务，BART模型的编码器和解码器的输入相同，但是将解码器最终时刻的隐藏层状态作为输入文本的语义向量表示，并利用线性分类器进行标签预测。利用标注数据和模型输出结果对模型参数进行调整。整个过程与BERT模型类似，在句子末尾添加特殊标记，利用该位置所对应的隐藏层状态表示文本。

对于生成式的任务，比如生成式文本摘要（Abstractive Summarization）、生成式问答（Abstractive Question Answering）等任务，精调时模型输入为任务所给定的输入文本，解码器所产生的文本与任务的目标文本构成学习目标。

对于机器翻译任务，由于其输入文本和输出文本是两种不同的语言，使用的不同词汇集合，因此不能采用与生成式任务相同的方法。为了解决上述问题，研究人员们提出了将BART模型的输入层前增加小型Transformer编码器，将源语言文本映射到目标语言的输入表示空间。同时，为了解决两段模型训练过程不匹配的问题，采取分阶段的训练方法。



在预训练阶段，大规模的数据使预训练语言模型有效地学习到了语言的通用语义表示，微调（Fine-tuning）则是利用预训练语言模型的主要范式，其目的是基于学习到的通用表示，针对目标领域的特性对模型进行调整，使其更适合下游任务。

相较于深入了解下游任务的特有知识，为其精心设计特别的模型，预训练模型只用转换下游任务的输入输出形式后进行微调，即可获得相当有竞争力的性能。



单句文本分类是自然语言处理中最为常见任务之一，其目的是判断一段文本所属的类别。

例如：判断一段电影评价的情感倾向是正面还是负面，判断一篇新闻所属的类别。

使用BERT进行单句文本分类任务时，对于将要进行单句文本分类的句子，BERT首先使用WordPiece进行分词，得到分词后的句子 $w_1w_2...w_n$ ，分别添加特殊字符[CLS]和[SEP]到句首和句尾，再经过输入层将其转换为BERT编码层所需的输入表示，其过程可以表述如下：

$$w = [\text{CLS}]w_1w_2...w_n, [\text{SEP}]$$

$$v = \text{InputLayer}(w)$$

随后，输入表示进入编码层，经过多层Transformer编码，每个位置的表示都通过自注意力机制进行充分交互，在最后一层得到具备丰富上下文信息的表示 h 。和预训练阶段时使用[CLS]进行NSP任务类似，在进行文本分类时，模型使用[CLS]位置的隐藏层表示进行预测。

$$H = \text{BERT}(V)$$

$$P = \text{Softmax}(h_{[\text{CLS}]}W + b)$$



句子对分类需要预测一对有关联的句子的类别

例如：判断一个句子的意思是否蕴含在另一个句子之中。

句子对分类与单句分类的区别在于处理的输入不同，BERT处理这两个任务时，也主要在输入上有所区别。

对于分词后的两个句子 $w_1^{(1)}, w_2^{(1)}, \dots, w_n^{(1)}$ 和 $w_1^{(2)}, w_2^{(2)}, \dots, w_m^{(2)}$ BERT使用[SEP]作为分隔符，将两个句子拼接到一起，在输入层转换为输入表示。

$$w = [\text{CLS}]w_1^{(1)}w_2^{(1)}\dots w_n^{(1)}[\text{SEP}]w_1^{(2)}w_2^{(2)}\dots w_m^{(2)}[\text{SEP}]$$

$$v = \text{InputLayer}(w)$$



序列标注任务需要解决的是字符级别的分类问题，其应用范围非常广泛，可用于分词，词性标注和命名实体识别等自然语言处理基础任务。以命名实体识别为例，在用序列标注的形式完成该任务时，需要对输入文本中的每一个词预测一个相应的标签，再根据整个序列的标签抽取出句子中的实体词。

传统的序列标注方法通常以词为输入的最小粒度，而在使用BERT等预训练模型时，**通常会使用分词器将词分割为更小粒度的子词**，这会破坏序列标注中词和标签一对一的关系。为了处理这种情况，可以让一个词的所有子词都保持原标签，或者只让第一个子词参与训练，预测时也只考虑第一个子词的预测结果。在完成分词后，将输入序列送入输入层转化为词向量，再将词向量送入预训练模型得到最终的隐藏层表示，其过程可以表示如下：

$$w = [\text{CLS}]w_1^{(1)}w_2^{(1)}\dots w_n^{(1)}[\text{SEP}]$$

$$v = \text{InputLayer}(w)$$

$$h = \text{BERT}(v)$$

在得到了隐藏层表示后，需要使用一个分类器对预测每个词在标签集上的概率分布：

$$P = \text{Softmax}(h_i W + b)$$

目录

Contents

6.1

语言模型概述

6.2

n元语言模型

6.3

神经网络语言模型

6.4

预训练语言模型

6.5

大规模语言模型

6.6

语言模型评价方法



受到**计算机视觉领域**采用ImageNet对模型进行一次预选训练，使得模型可以通过海量图像充分学习如何提取特征，然后再根据任务目标进行模型精调的范式影响，自然语言处理领域基于预训练语言模型的方法也逐渐成为主流。

ELMo为代表的动态词向量模型开启了语言模型预训练的大门，此后以GPT和BERT为代表的基于Transformer的大规模预训练语言模型的出现，使得自然语言处理全面进入了**预训练微调范式**新时代。

将预训练模型应用于下游任务时，**不需要了解太多的任务细节，不需要设计特定的神经网络结构**，只需要“微调”预训练模型，即使用具体任务的标注数据在预训练语言模型上进行监督训练，就可以取得显著的性能提升。



自2020年Open AI发布了包含1750亿参数的生成式大规模预训练语言模型GPT 3（Generative Pre-trained Transformer 3）以来，包含Google、Meta、百度、智源等公司和研究机构都纷纷发布了包括PaLM、LaMDA、T0等为代表的不同**大规模语言模型（Large Language Model, LLM）**，也称**大模型**。

表 6.2 典型大规模语言模型汇总

模型名称	参数量	训练单词数	研发机构	发布时间
ChatGPT	1750 亿	3000 亿	OpenAI	2022 年 11 月
Galactica	1200 亿	4500 亿	Meta AI	2022 年 11 月
BLOOMZ	1760 亿	3660 亿	BigScience	2022 年 11 月
U-PaLM	5400 亿	7800 亿	Google Research	2022 年 10 月
CodeGeeX	130 亿	8500 亿	清华大学	2022 年 9 月
PaLM	5400 亿	7800 亿	Google Research	2022 年 4 月
ERNIE 3.0 Titan	2600 亿	–	Baidu	2021 年 12 月
FLAN	1370 亿	–	Google	2021 年 9 月
GPT-3	1750 亿	3000 亿	OpenAI	2020 年 5 月
T5	110 亿	340 亿	Google	2019 年 10 月
RoBERTa	3.55 亿	22000 亿	Meta AI	2019 年 7 月
GPT-2	15 亿	100 亿	OpenAI	2019 年 2 月
BERT	3 亿	1370 亿	Google	2018 年 10 月
GPT-1	1 亿	–	OpenAI	2018 年 6 月

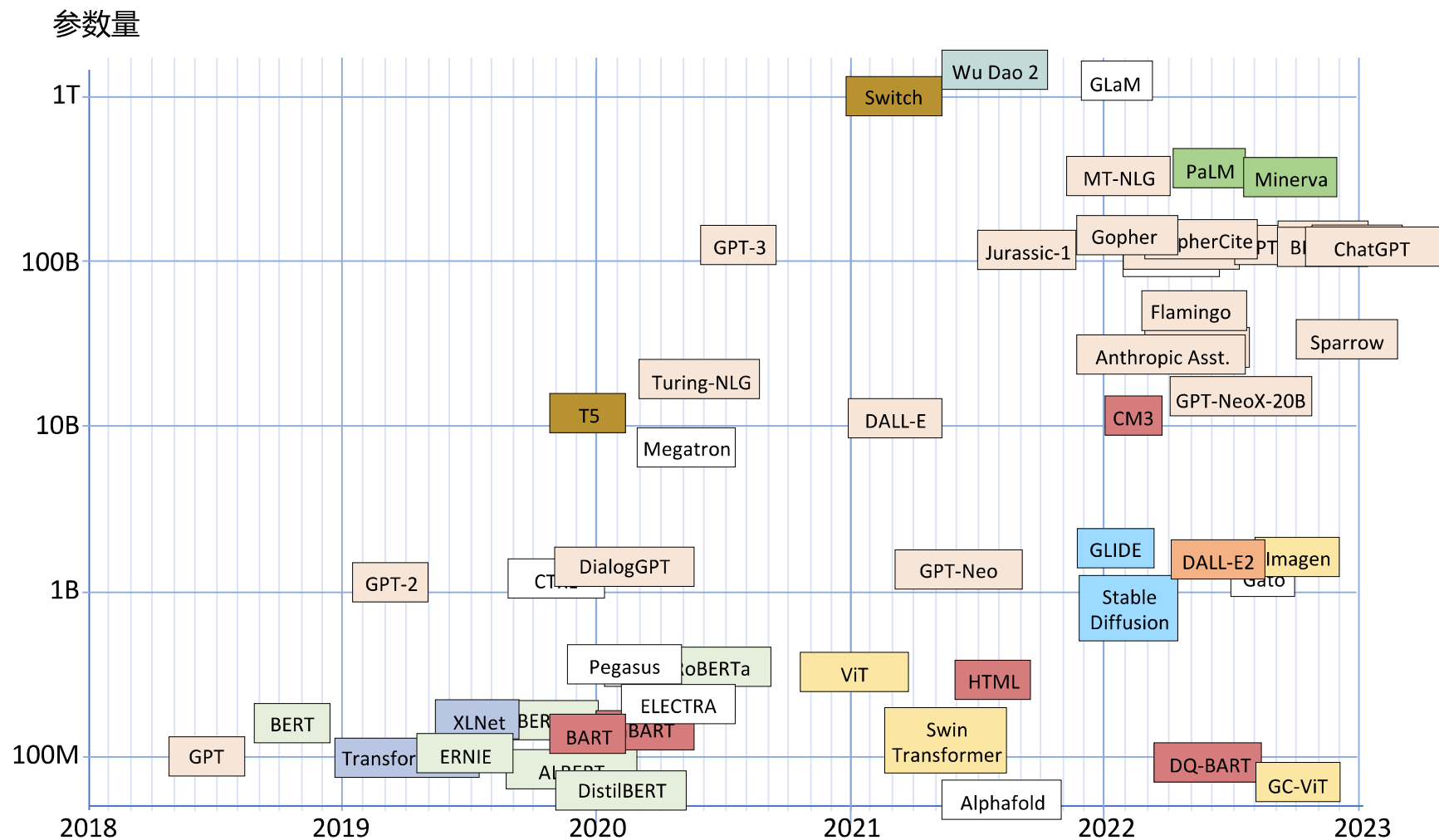
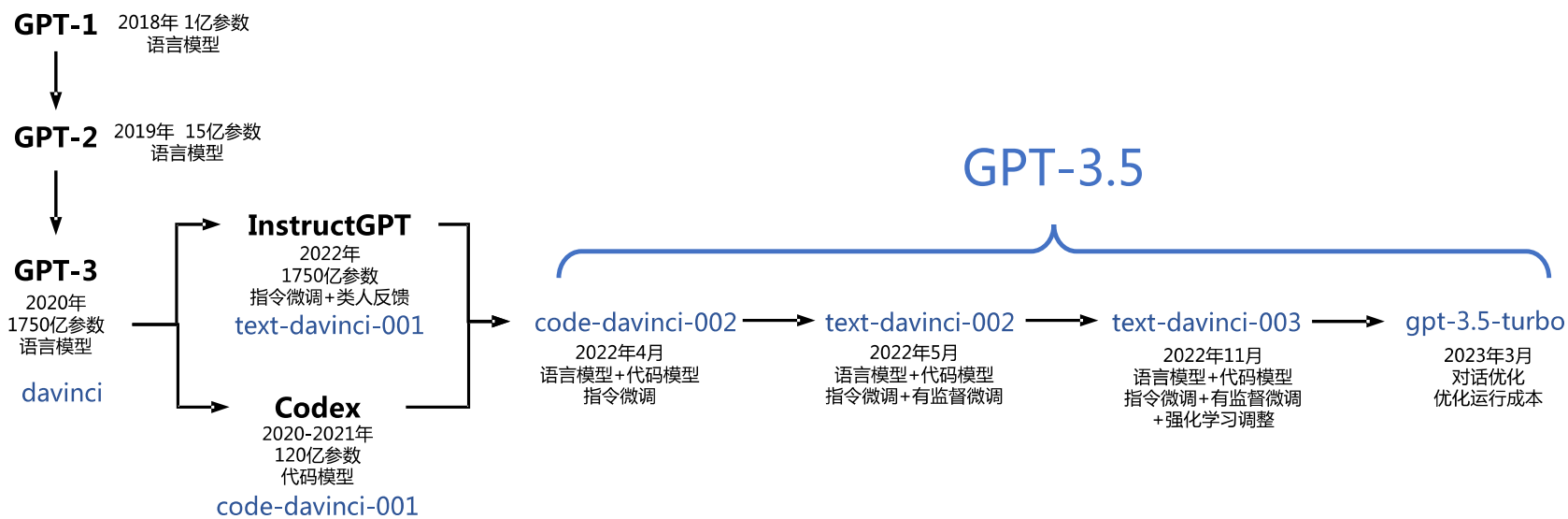


图 6.8 大模型时间线



2022年11月ChatGPT（Chat Generative Pre-trained Transformer）自发布起就引起了极大的关注，5天内注册用户超100万，在系统推出仅两个月后，月活跃用户估计已达1亿，并与Bing深度搜索结合构造了对话式搜索新范式。

ChatGPT整体过程可以分为三个阶段：**第一个阶段是基础大模型训练**，该阶段主要完成长距离语言模型的预训练，通过代码预训练使得模型具备代码生成的能力；**第二阶段是指令微调（Instruct Tuning）**，通过给定指令进行微调的方式使得模型具备完成各类任务的能力；**第三个阶段是类人对齐**，加入更多人工提示词，并利用有监督微调并结合基于强化学习的方式，使得模型输出更贴合人类需求。





文献[40]介绍了GPT-3模型的训练过程，包括模型架构、训练数据组成、训练过程以及评估方法。由于GPT-3并没有开放源代码，根据论文直接重现整个训练过程并不容易，因此文献[307]介绍了根据GPT-3的描述复现的过程，并构造[开源了系统OPT](#)（Open Pre-trained Transformer Language Models）。

在模型架构方面不论是GPT-3还是OPT所采用的模型结构都与我们在本章第6.4.2节所介绍的GPT-2模型一样，都采用由多层Transformer组成的单向语言模型，采用自回归方式从左到右对文本序列建模。但是针对不同的规模的参数量要求，其所使用的层数、自注意力头数、嵌入表示维度大小等具体参数各不相同。OPT给出了8种模型参数的细节。采用AdamW优化器进行优化，其参数 (β_1, β_2) 设置为\$(0.9, 0.95)\$。

表 6.3 OPT 不同模型规模下的具体参数细节^[307]

参数规模	层数	自注意力头数	嵌入向量维度	学习率	全局批次大小
125M	12	12	768	$6.0e-4$	50 万
350M	24	16	1024	$3.0e-4$	50 万
1.3B	24	32	2048	$2.0e-4$	100 万
2.7B	32	32	2560	$1.6e-4$	100 万
6.7B	32	32	4096	$1.2e-4$	200 万
13B	40	40	5120	$1.0e-4$	400 万
30B	48	56	7168	$1.0e-4$	400 万
66B	64	72	9216	$0.8e-4$	200 万
175B	96	96	12288	$1.2e-4$	200 万



在预训练语料集方面，根据文献[40]中的报道，**GPT-3中通过主要包含**经过过滤的Common Crawl数据集、WebText2、Books1、Books2以及英文Wikipedia等数据集合。其中CommonCrawl的原始数据有45TB，进行过滤后仅保留了570GB的数据。通过子词方式对上述语料进行切分，大约一共包含**5000亿子词**。为了保证模型使用更多高质量数据进行训练，在GPT-3训练时，根据语料来源的不同，设置不同的采样权重。在完成3000亿子词训练时，英文Wikipedia的语料平均训练轮数为3.4次，而Common Crawl和Books 2仅有0.44次和0.43次。

由于Common Crawl数据集合的过滤过程繁琐复杂，**OPT则采用了**混合RoBERTa、Pile和PushShift.io Redit数据的方法。由于这些数据集合中包含的绝大部分都是英文数据，因此OPT也从Common Crawl数据集中抽取了部分非英文数据加入训练语料。



由于模型参数量和所使用的数据量都非常巨大，普通的服务器单机无法完成训练过程，因此通常采用分布式架构完成训练。GPT-3和OPT中没有对这个部分给出详细的描述。

- 文献[40] 针对**GPT-3**的训练过程仅介绍了训练过程全部使用**NVIDIA V100 GPU**
- 文献[307]介绍了**OPT**使用了**992块NVIDIA A100 80G GPU**，采用全分片数据并行（Fully Shared Data Parallel以及Megatron-LM张量并行（Tensor Parallelism，整体训练时间将近2个月。
- **BLOOM**则公开了更多在硬件和所采用的系统架构方面的细节。该模型的训练一共花费3.5个月，使用48个计算节点。每个节点包含**8块NVIDIA A100 80G GPU（总计384GPU）**，并且使用4 NVLink用于节点内部GPU之间通信。节点之间采用四个Omni-Path 100 Gbps网卡构建的增强8维超立方体全局拓扑网络通信。



BLOOM使用Megatron-DeepSpeed框架进行训练，主要包含两个部分：Megatron-LM提供张量并行能力和数据加载原语；DeepSpeed提供ZeRO优化器、模型流水线以及常规的分布式训练组件。通过这种方式可以实现数据、张量和流水线三维并行。

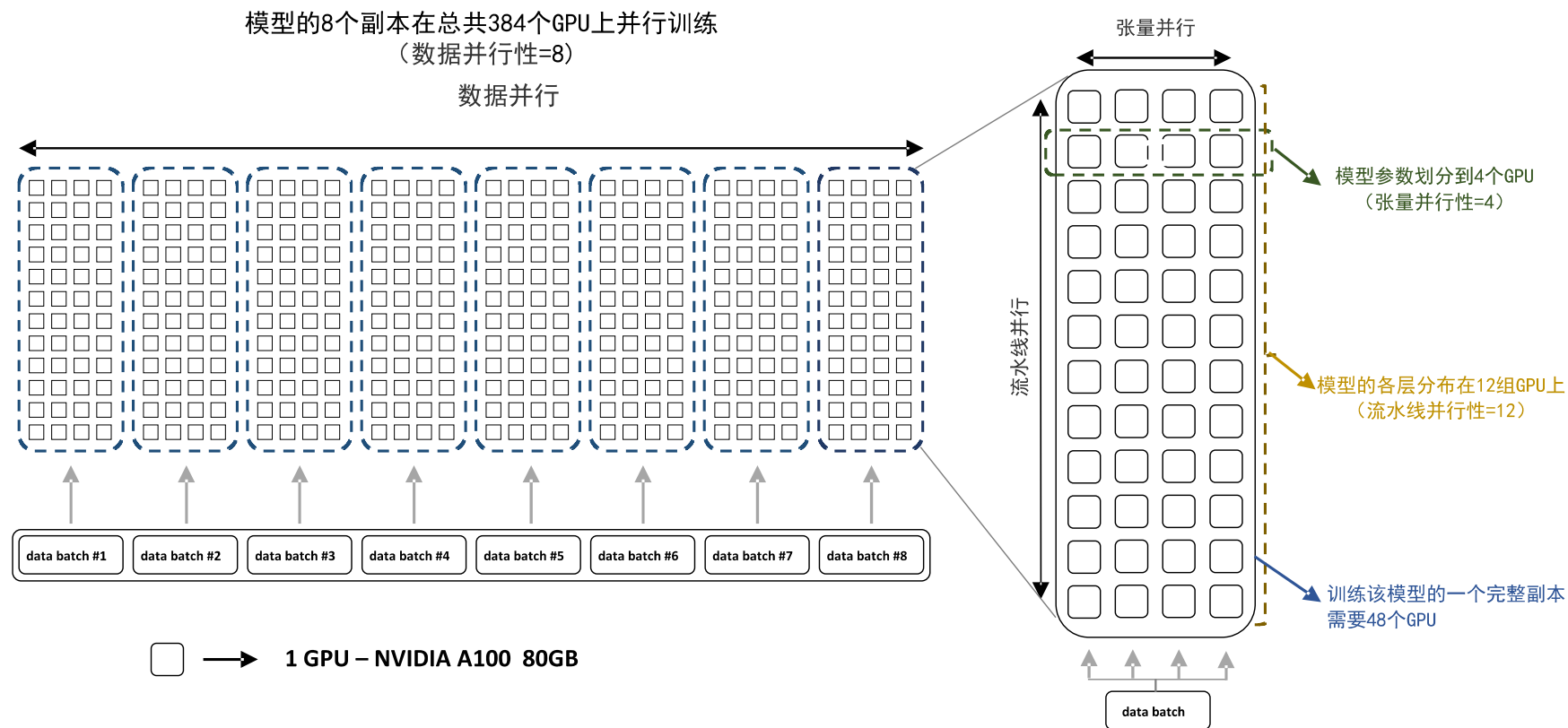


图 6.10 BLOOM 并行结构



以BERT为代表的预训练语言模型需要根据任务数据进行微调（Fine-tuning），这种范式可以应用于参数量在几百万到几亿规模的预训练模型。但是针对数十亿甚至是数百亿规模的大模型，针对每个任务都进行微调的计算开销和时间成本几乎都是不可接受的。因此，研究人员们提出了**指令微调（Instruction Finetuning）方案**，将大量各类型任务，统一为生成式自然语言理解框架，并构造训练语料进行微调。

例如，可以将情感倾向分析任务，通过如下指令，将贬义和褒义的分类问题转换到生成式自然语言理解框架：

For each snippet of text, label the sentiment of the text as positive or negative.

Text: this film seems thirsty for reflection, itself taking on adolescent qualities.

Label: [positive / negative]



利用有标注数据集合，再结合上述指令模板，就可以生成大量用于微调的训练数据。利用这些训练数据，就可以在一个大模型中同时训练大量不同的任务。当前的研究工作表明，这种训练方法可以使得模型具有很好的**任务泛化能力**，很多没有出现在指令微调训练语料中的任务也可以很好的完成，在零样本和少样本的情况下获得非常好的任务性能。

FLAN-T5\upcite{chung2022scaling}中通过混合之前的Muffin、T0-SF、NIV2以及思维链（Chain-of-thought, CoT）混合微调等工作，使用**473个数据集合**，针对**146个任务类型**，构造了**1836个任务**。数据集合包括SQuAD、MNLI、CoNLL2003等。任务类型包括阅读理解、问题生成、常识推理、对话上下文生成、完型填空、命名实体识别、文本分类等。



指令
不包含样例

训练目标不包含思维链

Answer the following
yes/no question.

Can you write a whole
Haiku in a single tweet?



yes

指令
包含样例

Q: Answer the following
yes/no question.
Could a dandelion suffer
from hepatitis?

A: no



yes

Q: Answer the following
yes/no question.
Can you write a whole Haiku
in a single tweet?
A:

训练目标包含思维链

Answer the following yes/no question
by reasoning step-by-step.

Can you write a whole Haiku in a
single tweet?



A haiku is a japanese
three-line poem. That
is short enough to fit in
280 characters. The
answer is yes.

Q: Answer the following yes/no question by
reasoning step-by-step.
Could a dandelion suffer from hepatitis?
A: Hepatitis only affects organisms with livers.
Dandelions don't have a liver. The answer is no.



A haiku is a japanese
three-line poem. That
is short enough to fit in
280 characters. The
answer is yes.

Q: Answer the following yes/no question by
reasoning step-by-step.
Can you write a whole Haiku in a single tweet?
A:

图 6.11 指令和训练目标中思维链示例



经过指令微调后的模型，虽然在开放领域任务能力表现优异，但是模型输出的结果通常是简单答案，这与人类的回答相差很大。因此需要进一步优化模型，使其可以生成更加贴近人类习惯的文本内容。

由于自然语言处理语料集合中所包含的答案，通常都是简短的标签，基本没有类似人类回答的有监督数据。如果全部使用人工，将自然语言处理任务数据集进行改造，构造类似人工回答的训练语料，所需要的规模过于庞大，时间成本和人工成本都过于高昂。

Open AI提出了使用**基于人类反馈的强化学习方法**（Reinforcement Learning from Human Feedback, RLHF），从而大幅度降低了数据集构建成本，但是达到了非常好的效果。



RLHF算法主要分为如下三个步骤：

- (1) 收集初始数据初步训练大模型，从得的指令集合中采样部分数据，对初始的大模型进行有监督微调；
- (2) 收集打分数据并训练奖励（Reward）函数，使用无标注的指令数据，收集模型的多个输出结果，评价人员根据模型的输出结果进行对比评价，确定模型输出的结果排序，并利用该评分训练奖励函数，使其未来可以对模型输出的优劣进行判断；
- (3) 使用更多的指令数据，利用奖励函数输出的得分，利用强化学习机制根据奖励得分进一步优化大模型。

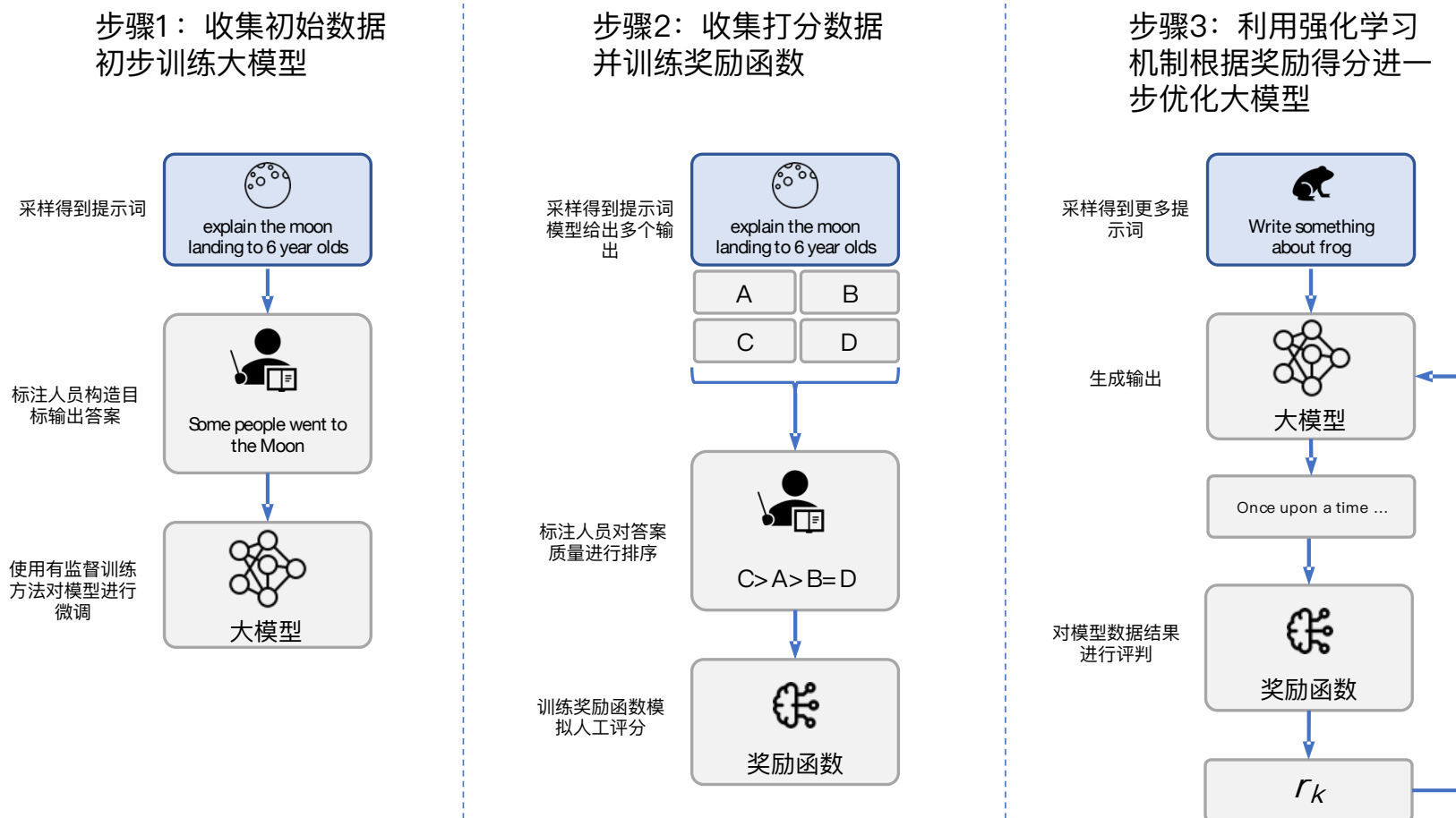


图 6.12 基于人类反馈的强化学习方法基本步骤

目录

Contents

6.1

语言模型概述

6.2

n元语言模型

6.3

神经网络语言模型

6.4

预训练语言模型

6.5

大规模语言模型

6.6

语言模型评价方法



语言模型最直接的测评方法就是使用模型计算测试集的概率，或者利用**交叉熵（Cross-entropy）**和**困惑度（Perplexity）**等派生测度。

对于一个平滑过的概率 $P(w_i|w_{i-n+1}^{i-1})$ 的n元语法模型，可以用下列公式计算句子P(s) 的概率：

$$P(s) = \prod_{i=1}^n P(w_i|w_{i-n+1}^{i-1})$$

对于由句子(s₁,s₂,...,s_n)组成的测试集T，可以通过计算T中所有句子概率的乘积来得到整个测试集的概率：

$$P(T) = \prod_{i=1}^n P(s_i)$$

交叉熵的测度则是利用预测和压缩的关系进行计算。对于n元模型 $P(w_i|w_{i-n+1}^{i-1})$ ，文本s的概率为P(s)，在数据s上n元模型 $P(w_i|w_{i-n+1}^{i-1})$ 的交叉熵为：

$$H_p(s) = -\frac{1}{W_s} \log_2 P(s)$$

其中，W_s为文本s的长度，该公式可以解释为：利用压缩算法对s中的W_s个词进行编码，每一个编码所需要的平均比特位数。

困惑度的计算可以视为模型分配给测试集中每一个词汇的概率的几何平均值的倒数，它和交叉熵的关系为：

$$PP_s(s) = 2^{H_p(s)}$$

交叉熵和困惑度越小，语言模型性能就越好。不同的文本类型其合理的指标范围是不同的，对于英文来说，n元语言模型的困惑度约在50到1000之间，相应的，交叉熵在6到10之间。

本章小结

在本章中，我们介绍了语言模型的基础概念，并介绍了n元语言模型、神经网络语言模型、预训练语言模型以及大规模语言模型。预训练语言模型正逐渐成为自然语言处理的基础模型。预训练语言模型通过设计特定的自监督训练目标，有效地从大量标记和未标记数据中获取知识，并存储到巨大的参数中。通过在特定任务上进行微调，隐藏在巨大参数中的丰富知识可以使各种下游任务受益。随着以GPT3为代表的大规模语言模型在各类型自然语言处理任务上取得的成功，大模型的研究已成为当前的研究热点，也许会成为未来的新的研究范式。